__/__/__

Week-10 DL Gen AI

L1 From RNN Atten^n to Transformer Atten^n

RNN Atten^n → works as a prototype for Gen AI

Transformer Atten^n → key for Gen AI (universal architecture) works on any NN.

- So far – RNN process sequences, atten^n focuses on relevant parts, & scaled dot-prod. atten^n. diff. btw. self-atten^n & cross-atten^n
  → challenge → can't move ahead until prev. is processed. Thus, making training slow & long-range dependencies difficult.

⇓

what if remove RNN & just keep atten^n.

Transformer.

→ parallel processing of all posi^n
→ self-atten^n is the core opera^n
→ direct interac^n btw. any 2 tokens.
→ O(1) parallel depth
→ scales well with data + hardware.

- Scale → Billions → Trillions.
- context → 512 → 1M+ tokens
- allow multimodality (text, images, etc.)
- emergent abilities → reasoning, planning, etc.
- Key idea → 1 architecture + scale + data = general intelligence

__/__/____

- Positional encoding → add a special vector c/d a positional encoding to each word's embeddings. It is added before feeding the words into transformer.
  - → Learnt embeddings → assume some vector & learn it.
  - → Sinusoidal embeddings → we give it ourselves
    - ↳ $[\vec{e_i} + \vec{p_i}]$, like add a timestamp

→ Let p be the posi^n of a token in seq. ($p = 0, 1 \ldots$) & i be the dim. index of the embedding ($i = 0, 1 \ldots d-1$)

$$PE_{(p,2i)} = \sin\left(\frac{p}{10000^{2i/d}}\right) \Rightarrow \text{even dim.}$$

$$PE_{(p,2i+1)} = \cos\left(\frac{p}{10000^{2i/d}}\right) \Rightarrow \text{odd dim.}$$

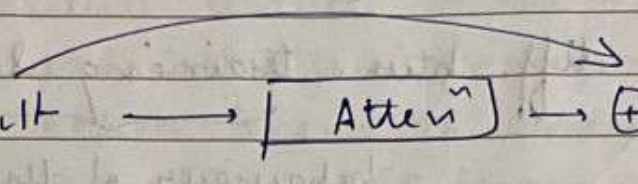
here $2i$ & $2i+1$ span the dimensions from 0 to $d-1$.

→ some cater to low & some to high freq.
→ it allows the model to easily learn to attend based on the relative positions.
→ the denominator term causes the wavelength of the sinusoids to vary across the embedding dimensions.

- A transformer is a deep stack of attn layers
  2 components are crucial.
  - → residual or skip connects ⇒ gradient gateway
  - → layer normaliza^n
    - ↳ nos. should not blow up or disappear.

__/__/____

→ Residual connec^{ns} –



so we don't miss out on the infoerma^{n}.

→ Layer Noerm → after residual connec^{n}. It rescales the nos. in the vector to have a std. noarmal distribu^{n}.

① calc. mean → $\mu = \frac{1}{H}\sum_{i=1}^{H} x_i$ (mean of all elements in the vector.)

② calc. var → $\sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2$

③ Noerm → $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}$ (ε for numerical stability)

④ scale & shift → $y_i = \gamma_i \hat{x}_i + \beta_i$ (γ: gain, β: bias)

→ these calculations are done idadependentty.

→ adv.

1. stabiliy training
2. enables deep stacking
3. fasten convergence.

__/__/____

L3 The Original Transformer

- there is diff. btw. training & inference.
  behaviour of the transformer is different.
- each token in RNN is processed sequentially.
- right now, encoder has the full data available to us. So, we can process them parallelly.
- In the decoder,
  Training → entire target sentence is available.
  Simultaneously can be done.
  we used masked - MHA, teacher - force
  (Auto-regressive (decoder in action))
  Inference → target seq. isn't available to us.
  ↳ it has to run sequentially. No other option for transformers as well.

- transformer has a classical enc-dec. struc.
  → Encoder → reads the entire input seq., builds a rich, context-aware representation of each token. consist of a stack of identical enc. layers.
  → decoder → generates the output seq. 1 token at a time (during inference). It attends to the encoder's output to get context from the source sentence. Again a stack of identical decoder layers. (has both self & cross attention)
  → here we rely entirely on self-attention instead of recurrence.

__/__/____

+ MHA (multi-head attenⁿ) - sometimes a single attenⁿ mechanism might get confused.
  (↳ equivalent to multiple filters in the CNN.
  ↳ each attenⁿ head looks at diff. types of patterns (eg. syntax, semantics, pronouns, etc.)
→ it (all attenⁿ heads) runs parallely.
→ steps
  1. input is split & sent to each 'head'.
  2. each head performs its own attenⁿ calc. in parallel.
  3. results from all heads are concatenated.
  4. finally, a linear layer combines the results into a single output.
→ each of the heads' weights are initialized randomly & then the backprop. learns them.

- Mechanism:
→ each head $i \in \{1 \dots h\}$
$Q_i = Q W_i^{(Q)}$, $K_i = K W_i^{(K)}$, $V_i = V W_i^{(V)}$
→ $head_i = attenⁿ(Q_i, K_i, V_i)$
→ concat $(head_1, \dots head_h)$
→ Multihead (Q, K, V) = concat (heads) $W^{(O)}$
↓ linear layer wts.

L2 Parts of the Transformer Architecture

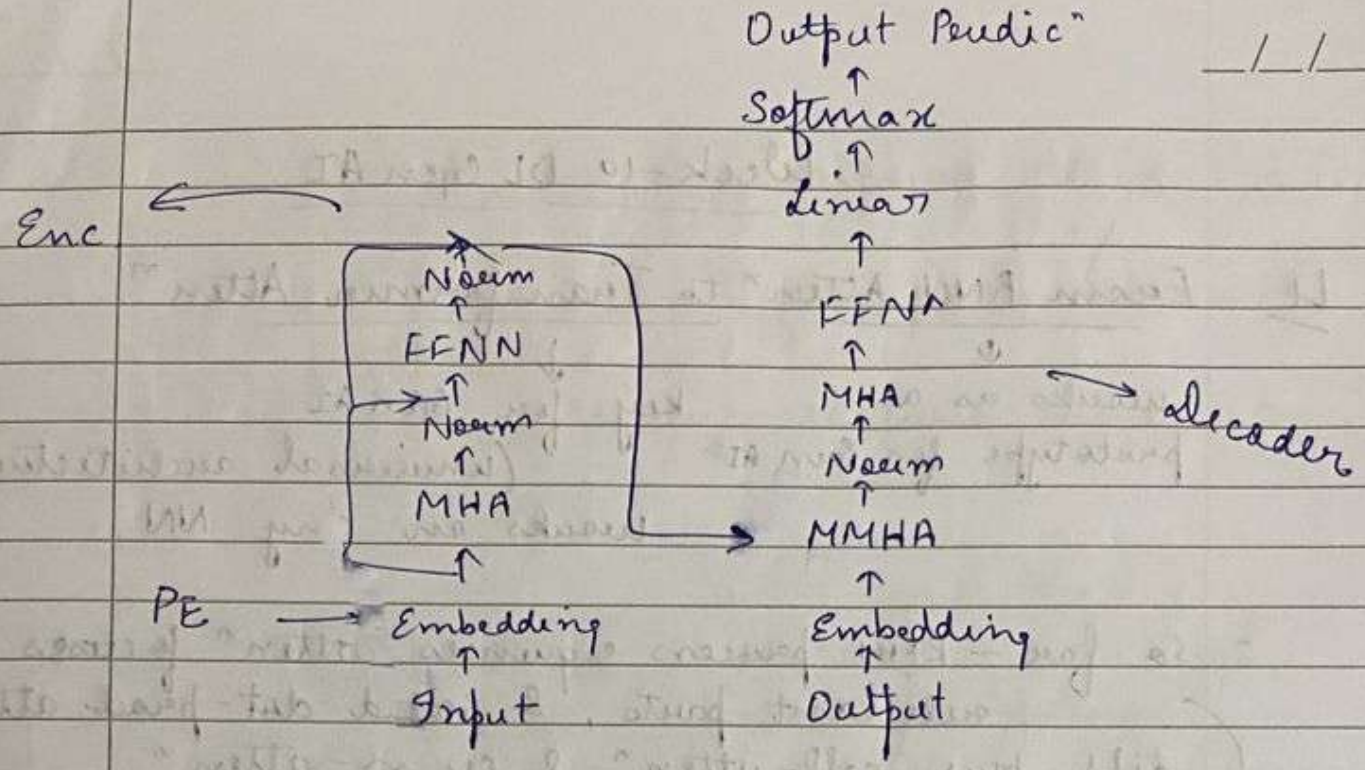
- self-attenⁿ is permutaⁿ invariant. It looks like bag of words with no order.

__/__/____

- Encoder layers
  → has residual connec^n + layer norm
  → multihead self attein (MHA) → look at all other tokens to understand context.
  → positionwise FFN → FC layer applied independently to each token's representa^n.
- decoder layers
  → masked self atten^n (MMHA) → prevents the decoder from looking at future tokens in the output seq. it is trying to predict.
  → Enc-dec cross atten^n → Q from dec. & K & V comes from enc. output.
  → positionwise FFN
- Data Flow -
  1. input embeddings
  2. Positional embeddings
  3. encoder stack → refine the representa^n
  4. decoder stack → MMHA & then cross-atten^n to take encoder outputs.
  5. final linear layer & softmax.
     ↳ acts as the classifier → probab. distribu^n over the entire vocabulary for the next token.

L4 From Transformers to LLMs

- Enc. only - BERT, ROBERTa → understanding, classifica^n
  dec. only - GPTs, LLaMa, Claude → Genera^n, chat



- Understanding self-attenⁿ

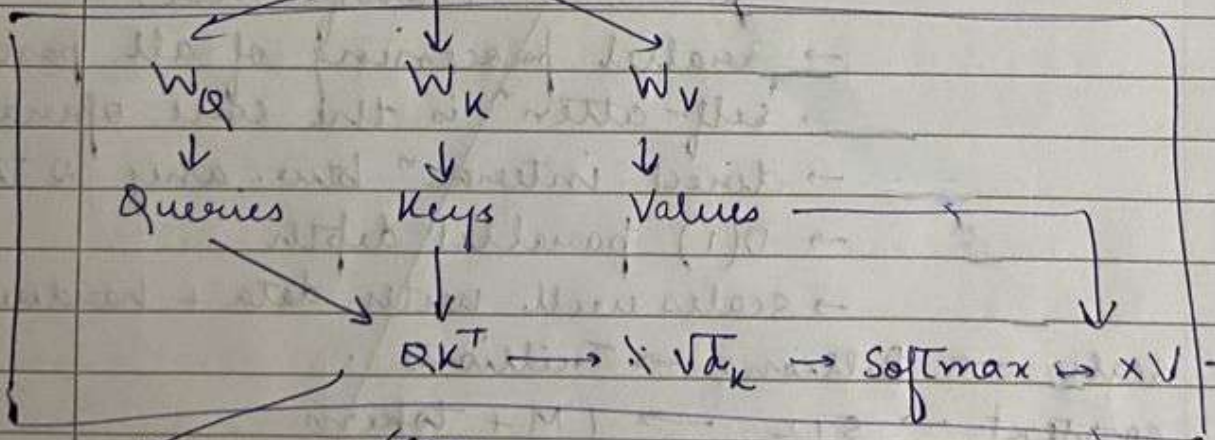
[—, —, —]

↓

Embedding

↓

Token



how much attenⁿ to pay to myself & others.

we get the modified embeddings (it interprets the entire sentence context)

__/__/____

- Enc. only models
  → has bidirectional context
  → pre-training objective : MLM (masked lang. modelling)
    ↓
    model predicts randomly masked words in a sentence. [MASK]
  → produces a high quality embeddings for each input token, which can be used by a classificaⁿ layer.
  → sentiment analysis, NER, QA, sentence classificaⁿ.
  → Best eg. → BERT (Bidirectional Encoder Representations from Transformers)

- Dec. only models
  → foundaⁿ of modern generative AI
  → unidirectional (L→R) causal context
  → pre-training obj. : Autoregressive lang. modelling
    ⇓
    predicts next token
  → generates a probability distribuⁿ over the entire vocab for the next token.
  → chatbots, text summarizaⁿ, content genera", code genera".

- first pre-train the model with entire data to understand the fundamentals. Then just train on small set of dataset (specific for the task) to fine-tune the model.

this is the general prac. now.

__/__/____

Enc-dec. - T5, BART — Transla", Summariza"
- Dec only dominates the market. Simpler architectures, scales better with data & compute, autoregressive genera":
→ removed enc. & cross-atten". Keep MSHA + ffNN, stack many such layer. There is no sep. source / target. Everything is just continua". Everything works in text genera".
- from small transformers to massive LLMs.
- scaling law → performance improves predictably with more params., training data, compute (GPU hours)
- old paradigm → train a model from scratch for each task.
- new → Transfer learning
1 → Pre-Training (expensive, once)
→ train on massive raw-text
→ predict next-token
→ model learns lang., facts, reasoning
2 → Fine-tuning (cheap, repetitive)
→ adapt to tasks
→ needs less data
→ can train using RLHF method
- Tokeniza"
→ not word level → BPE, WordPiece, SentencePiece
→ handles rare words, multiple lang., code
- Massive Training data
→ web crawls, trillions of tokens
→ careful filtering & clea".
- Training Infrastructure
→ 1000s of GPUs, months of training time
→ distributed training, precision.

__/__/____

- the problem with base models, it only predicts next token → should be continual, & not harmful
- Alignment Techniques
  → Instruc^n Fine Tuning (SFT) → train on response - instruc^n pairs.
  → RLHF → human rank model outputs & train reward model optimized using RL. (PPO)

  ⇓

  models that are helpful, harmless & follow instructions.
- now, we have emergent learning using few-shot learning, CoT reasoning, basic arithmetic, code generat^n, translat^n to many languages.
- more compute + more data + bigger model

  quantitatively more capabilities

L5 Modern Transformers in LMs

- The original transformer was perfect for seq2seq tasks like machine translat^n. Many tasks do not eg. full structure.
- Enc. only model
  → to understand the input, sees full context
  → Eg. BERT, used in natural lang. understanding
- dec. only model
  → generates an output, unidirectional
  → Eg. GPT, natural lang. generat^n.
- Let's study them in depth.

__/__/__

- BERT
  → uses MLM (15% tokens are masked) & NSP (next sentence predict") for pre-training.
  ↓ helps to understand sentence relationships
  → This eventually led to RAG. (Retrieval Augmented Gen").
  → very powerful for lang. understanding & document processing.
- GPT (Generative Pre-Trained Transformer)
  → autoregressive (L → R)
  → pre-trained on std. LM → predicts the next token.
  → inherently generative ⇒ creative in thoughts & writing

-	BERT	GPT
	→ enc.	→ dec.
	→ bidirectional	→ unidirectional
	→ MLM	→ CLM
	→ NLU	→ gen"